

AI Tools & Usage Documentation

Detailed Distinction of Development vs. Runtime AI, Evaluation Frameworks & System Analysis

DOCUMENT STATUS	ENGINEERING ROLE	RUNTIME LLM	EVALUATION STATUS
Final Official Documentation	AI Full-Stack Development	Google Gemini 3.5 Flash / Vision	9/9 Test Suites Passing (100%)

1. Executive Summary & Development Workflow

This document provides comprehensive transparency into the role of Artificial Intelligence in the **AI Study Companion** platform. To ensure complete clarity, this documentation strictly demarcates:

- AI Used to Build the Product:** The development coding assistants, debugging subagents, automated test generators, and design tools employed during engineering.
- AI Used by the Final Product:** The production runtime AI services (Grounded Tutor, Multimodal Vision OCR, Computer-Adaptive Testing, Rubric Grading, Knowledge Graph Extraction, and Failover Content Synthesis).

Development Process Statement: The entire product was engineered through disciplined pair programming with an advanced AI coding assistant. The human engineer provided **step-by-step instructions for each feature, implementing, validating, and testing one by one** in an incremental, agile workflow—starting with foundational database isolation, progressing through document chunking, multimodal handwriting OCR, RAG citation grounding, and finishing with computer-adaptive quizzes and administrative telemetry.

2. AI Used to Build the Product (Development Tools)

2.1 Coding Assistants & Development Agents

- Google Antigravity Agentic IDE:** Acted as the primary autonomous pair-programmer. Followed strict planning and execution cycles: researching existing code patterns, generating implementation plans, executing targeted file edits, and verifying through automated test runners.
- Layer-by-Layer Architectural Scaffolding:**
 - Backend:* Scaffolding Express controllers, Mongoose schemas, Multer streaming handlers, and asynchronous background worker queues.
 - Frontend:* Building dynamic React components with Vite, custom hooks, Server-Sent Events (SSE) streaming listeners, and SVG-based progress visualizations.

2.2 Debugging Tools & Subagents

- Automated Subagents for Log Tracing:** Deployed autonomous background tasks to capture live server logs, analyze Node.js process crashes, and diagnose HTTP 429 quota exhaustion spikes from external APIs.
- Regex Boundary Bug Isolation:** Automatically identified a critical regex bug in `retrievalScoring.js` where substring matches on words like *"combination"* or *"explanation"* falsely triggered queries containing *"nation"*, fixing it with exact word boundaries (`\b${tok}\b`).

- **Database Orphan & Migration Scripts:** Generated standalone diagnostic scripts (`reassign-orphans.js` , `set-single-admin.js`) to sanitize development MongoDB collections without manual database tampering.

2.3 Design & Styling Tools

- **Tailwind CSS v3 Design System:** Automated translation of UI mockups into clean, accessible utility classes adhering to curated dark-mode palettes (slate-900 backgrounds, indigo-600 accents, emerald-500 mastery highlights).
- **Responsive Layout Polish:** Ensured seamless transitions across mobile, tablet, and desktop viewports, eliminating horizontal scroll overflow and ensuring touch-friendly targets.

2.4 Testing & Benchmarking Agents

- **Synthetic 500-Page Textbook Benchmark:** Engineered automated test scripts (`test-500-pages-rag.js`) generating 500 dense technical pages (560,000+ characters) to mathematically verify that in-memory RAG retrieval executes under 60ms.
- **Automated Regression Suite:** Created `backend/tests/run-tests.js` spanning 9 integration suites verifying authentication, tenant isolation, grounded citations, out-of-domain refusals, and adaptive quiz generation.

3. AI Used by the Final Product (Runtime Capabilities)

Runtime AI Feature	Model / Engine	Production Architecture & Function
Grounded AI Tutor	gemini-3.5-flash	Generates in-depth explanations strictly from project evidence chunks. Streams tokens in real time via SSE; strips markdown double-asterisks (**); emits clickable page citations.
Multimodal Vision OCR	gemini-3.5-flash (Files API)	Transcribes scanned and handwritten notes verbatim in 15-page batches (<code>=== Page X ===</code>), capturing cursive text, SQL statements, and formulas.
Concept Discovery Engine	gemini-3.5-flash	Extracts atomic concepts from document chunks with descriptions, prerequisites, and initial baseline mastery levels.
Computer-Adaptive Testing	gemini-3.5-flash	Generates dynamic 7-question quizzes where Question $N+1$ difficulty adapts to Question N performance (Beginner, Intermediate, Advanced).
Rubric-Based AI Grader	gemini-3.5-flash	Evaluates student responses against 3-point rubrics (0–100 score), awarding partial credit and conceptual breakdown.
Model Failover System	gemini-3.1-flash-lite / gemini-2.5-flash	Automatically switches to candidate fallback models if the primary model encounters HTTP 429 rate limits or HTTP 503 capacity spikes.
Deterministic Content Engine	In-House Algorithmic Fallback	Guarantees 100% platform availability if all external AI APIs fail or network connectivity drops.

4. Evaluation Approach & Verification Framework

System performance, retrieval quality, and AI behaviors were evaluated through rigorous, reproducible benchmarks:

4.1 Retrieval Accuracy & Grounding Validation

- **Exact Page Ground-Truth Test:** Tested against a 24-page research document (`Invisible_Rules_Santhosh_FINAL_v4.pdf`). When querying *"what is in page 24"*, the RAG engine accurately extracted Page 24 (Section 9: Bibliography with 12 scholarly references) rather than earlier survey tables.
- **Multi-Page Intent Test:** Tested against a 41-page handwritten notebook (`SQL Handwritten Notes - 1.pdf`). When querying *"summarize pages 10 to 20"* or *"pages 26 to 41"*, the engine extracted chunks exclusively from the target page bounds with zero spillover.

4.2 Hallucination Prevention & Domain Refusal

- **Strict Out-of-Domain Refusal:** When a user asks an external question (such as *"father of nation"* or *"sourdough recipe"*) inside a project containing SQL or Machine Learning materials, the system identifies zero matching evidence chunks, sets `isUnsupported: true`, and explicitly replies:
"I could not find any information or evidence regarding '[query]' in your uploaded project materials... As your AI Study Companion, my responses are strictly restricted to your uploaded document(s)."

- **No External Fabrications:** The system strictly suppresses external world trivia and refuses to metaphorically connect unrelated political/historical figures to technical topics.

4.3 Performance & Scalability Benchmarks

- **500-Page Scale Benchmark (`test-500-pages-rag.js`):**
- Ingested 500 simulated textbook pages (560,000 characters) partitioned into 1,000 chunks in **40ms**.
- Uniform sampling extracted front matter, core chapters, and appendices in 25,000 characters without context window overflow.
- In-memory semantic search retrieved target chunks in **30ms to 57ms** across all 1,000 chunks.

5. Known Limitations

The platform's architectural trade-offs and operational boundaries are documented below:

- **External AI Quota & Rate Limits (HTTP 429):** Google AI Studio free-tier API keys enforce strict TPM (Tokens Per Minute) and RPM limits. While our automatic failover pipeline rolls over to `gemini-3.1-flash-lite` and the Content Engine, heavy concurrent classroom usage requires dedicated enterprise quotas.
- **Vision OCR Batching Latency:** Because massive 100-page scanned documents are processed in 15-page batches to prevent token exhaustion, full document transcription can require 1 to 3 minutes in the background worker queue.
- **In-Memory Background Queue:** The current worker queue runs in-memory within the Node.js process. In an enterprise multi-cluster deployment, this should be migrated to Redis + BullMQ for cross-instance persistence.
- **Encrypted & Password-Protected PDFs:** PDFs encrypted with user passwords cannot be parsed by standard stream layout engines or Gemini Vision without prior decryption.
- **Local Storage on Single Instance:** Uploaded original PDF binaries are stored on the local container filesystem. Production clustering requires object storage (e.g., AWS S3 or Google Cloud Storage).

6. Future Improvements

With additional engineering cycles, the following roadmap enhancements are scheduled for implementation:

1. **Distributed Queue with BullMQ & Redis:** Decouple document processing into dedicated container workers with auto-scaling and persistent retries.
2. **Hybrid Dense-Sparse Vector Search:** Augment in-memory token scoring with a persistent vector database (pgvector or Pinecone) combining BM25 keyword matching with dense embeddings.
3. **Interactive Code Sandbox & Math Rendering:** Embed Monaco Editor and KaTeX for real-time SQL execution and interactive mathematical derivation directly inside the Tutor chat.
4. **Audio Tutor via WebRTC & Gemini Live:** Integrate voice-to-voice streaming allowing students to have natural spoken conversations with their study companion.
5. **Spaced-Repetition Anki Export:** Automatically convert discovered concepts into `.apkg` flashcard decks incorporating the SuperMemo SM-2 spaced repetition algorithm.
6. **Instructor Analytics & Cohort Mastery Heatmaps:** Enable professors to assign reading materials to shared Spaces and view aggregate student mastery heatmaps across entire classes.